UNIVERSITY OF LEICESTER

CO2124

Muhammad Iftikhar

Lecture 9:
Mobile App Development (3)

UNIVERSITY OF LEICESTER

Announcement about the Class Test

- 25th & 26th March, 2026
- The test will be in-person closed book
- The test must be conducted in your scheduled lab timings
- Duration: 1 Hour. Once started the test must be completed
- This will be 30% on the module marks
- Venue: **Percy Gee Lab**
- There is no mock test since we have 4 quizzes already. The class test will follow the same format at the biweekly quizzes

UNIVERSITY OF LEICESTER

BB Announcement about the Class

Class Test The class test accounts for 30% of your overall semester mark. The test contains 45 questions, and the standard duration is one hour. Please read the essential information below carefully:

- **Students with Arrangements:** If you have a support plan or specific exam arrangements in place (e.g., extra time), we will contact you individually next week regarding your specific timing and location.

- **Group/Time Allocation:** Please attend according to your weekly lab session:
  - **Wednesday: 09:00 – 11:00 Labs:** Test runs 09:10 – 10:10.
  - **Wednesday: 11:00 – 13:00 Labs:** Test runs 11:10 – 12:10.
  - **Thursday: 09:00 – 11:00 Labs:** Test runs 09:10 – 10:10.
- *Please be seated in the lab at least 10 minutes before your start time. To avoid disruption, late arrivals will not be admitted.*

- **Location:** This test must be taken in the **Percy Gee labs only** during your allocated time. Attendance will be strictly monitored; submitting the test from outside the designated lab will be recorded as an absence.

- **Format:** The test will include MCQs, multiple-answer questions, fill-in-the-blank, and matching questions.

- **Academic Integrity:** This is a closed-book official university test. The use of mobile phones, textbooks, or the internet is strictly prohibited. Any violation of these rules will be regarded as cheating in accordance with university regulations.

- **Content Scope:** The test covers all material delivered this semester (CO2124), including all lab worksheets.

Previous Lecture:

- App Structure
- Activities
- Layouts
- Intents
- Views
- Android Toast

Today’s Lecture:

- Android App Performance and Optimisation
- Challenges in Android App Development
- Types of Data Storage in Android
- Android vs iOS

UNIVERSITY OF LEICESTER

Android App Performance and Optimisation

- **Performance** is crucial for a smooth user experience.
- Slow apps lead to poor user ratings, and customer dissatisfaction.
- Android apps must be optimised to deliver a responsive, efficient experience.
- Profiling tools and efficient coding practices play an important role in identifying and resolving performance issues.

Android App Performance and Optimisation

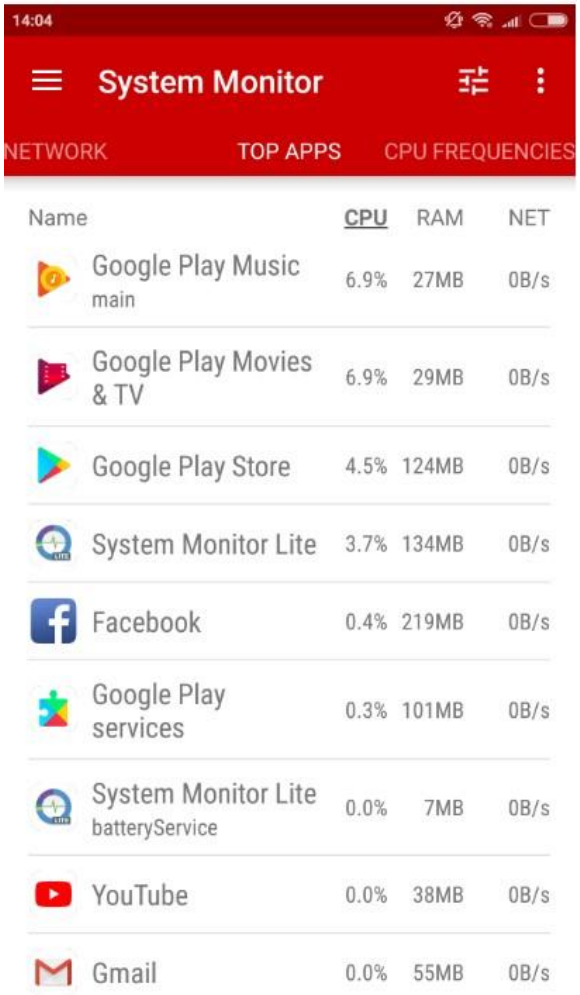
Common Performance Bottlenecks

- Long operations on the UI thread make the app unresponsive.
- Unused objects or resources are not cleared, consuming memory unnecessarily.
- Too many network calls can slow down the app and drain battery life.
- Heavy views and unoptimised layouts can slow rendering.
- Inefficient background operations consume CPU and battery.

Android App Performance and Optimisation

Optimising CPU Usage

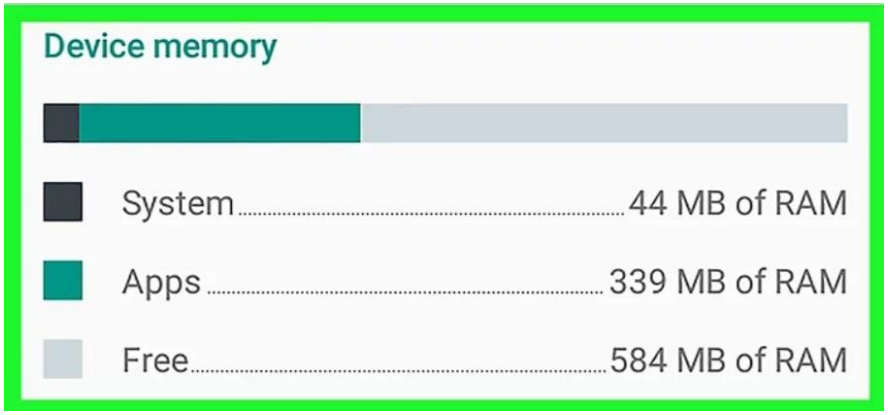
- Avoid CPU-heavy tasks on the main thread. Use background threads or AsyncTask.
- Use **Thread Pools** and ensure efficient usage of threads.
- Use Android's **Profiler** tool to monitor CPU usage and identify hotspots.



Android App Performance and Optimisation

Optimising Memory Usage

- Ensure objects are properly cleaned up to prevent memory leaks.
- Efficiently manage images to reduce memory consumption.
- Use Android's Memory Profiler to check heap usage and object allocations.



Android App Performance and Optimisation

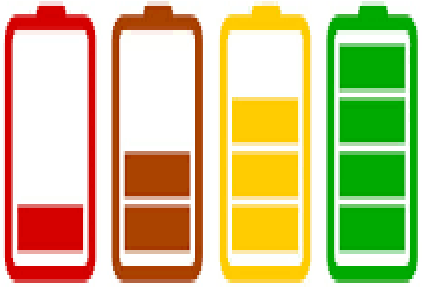
Improving Network Performance

- Minimise network request time by optimising APIs and using faster protocols (e.g., **HTTP/2**).
- Cache network responses to avoid redundant requests. Example: Use **Cache-Control** headers in HTTP responses for automatic caching.
- Minimise data transfer by reducing payload size (e.g., **JSON compression**, use **gzip**).

Android App Performance and Optimisation

Battery Optimisation

- Schedule background tasks efficiently to ensure they're done at the best times, without continuously waking the device. Example: Use **WorkManager** to schedule background tasks in a battery-friendly way.
- Avoid running services continuously in the background unless absolutely necessary. Example: Use **JobScheduler** instead of **Services** for periodic tasks.



Android App Performance and Optimisation

Optimising UI Performance

- Too many nested views can slow down rendering. Use **ConstraintLayout** for flatter hierarchies.
- Load only visible elements (e.g., use **RecyclerView** with lazy loading for large lists).
- Use lightweight layouts and avoid complex custom views unless needed. Example: Use **ConstraintLayout** or **FrameLayout** for simpler, more efficient layouts.
- Cache views that don't change frequently to avoid frequent re-inflation.

Android App Performance and Optimisation

Profiling Tools in Android

- **Android Profiler**: Monitors CPU, memory, and network usage in real-time.
- **LeakCanary**: A memory leak detection library that can help find memory leaks.
- **Systrace**: Captures detailed traces of your app's performance for deeper insights.

Challenges in Android App Development

Device Fragmentation

Android runs on a wide range of devices with different screen sizes, resolutions, hardware configurations, and OS versions.

Challenge:

- Ensuring compatibility across all devices can be difficult.
- Testing on various devices becomes complex.
- UI may look inconsistent on different screen sizes and resolutions.

Solution:

- Use Responsive Layouts and ConstraintLayout to make UIs adaptable.
- Test on multiple devices, using tools like Firebase Test Lab or Android Emulator for different device configurations.
- Minimise reliance on device-specific features to increase app compatibility.

Challenges in Android App Development

UI/UX Design Challenges

Designing a UI that works seamlessly across different screen sizes, orientations, and devices.

Challenge:

- Different screen resolutions and aspect ratios.
- UI elements can overlap or become too small/large on different screens.
- Ensuring a smooth and intuitive user experience.

Solution:

- Use ConstraintLayout and RelativeLayout for flexible and responsive designs.
- Implement multiple layout resources for different screen sizes.
- Focus on accessibility by ensuring the app is usable by people with disabilities.

Challenges in Android App Development

Handling Background Tasks and Multitasking

Challenge:

- Running background tasks without draining battery or using excessive CPU resources.
- Handling background tasks across different versions of Android (e.g., WorkManager vs AlarmManager).
- Managing long-running tasks like downloading data, syncing, or periodic operations.

Solution:

- Use WorkManager for reliable background task management across API levels.
- Use JobScheduler to schedule tasks based on network availability, battery status, etc.
- Use Foreground Services for tasks that need to continue running when the app is in the background.

Challenges in Android App Development

Security Challenges

Challenge:

- Sensitive Data Storage: Storing sensitive data securely (e.g., passwords, personal data).
- App Permissions: Managing app permissions to avoid excessive access to user data.
- Data Encryption: Securing data in transit and at rest.
- Malware and Hacking: Protecting the app from vulnerabilities and malicious attacks.

Solution:

- Use Android Keystore to store sensitive data securely.
- Implement HTTPS for secure data transmission and use SSL pinning to prevent man-in-the-middle attacks.
- Regularly update libraries and dependencies to mitigate security risks.

Challenges in Android App Development

App Compatibility and Updates

Challenge:

- API changes and new Android versions can break existing app features.
- Supporting a wide range of Android versions.
- Updating the app without causing regressions.

Solution:

- Test the app across different Android versions using Firebase Test Lab.
- Maintain multiple APKs for different screen densities and API levels or use Android App Bundles.

Challenges in Android App Development

Testing and Debugging in Android Development

Challenge:

- Testing on multiple Android versions, screen sizes, and devices.
- Identifying performance bottlenecks.
- Debugging crashes and issues that only occur in specific conditions.

Solution:

- Profile the app using **Android Profiler** to detect performance issues related to CPU, memory, and network usage.
- Test on a variety of real devices using Firebase Test Lab or Emulator for different configurations.

Data Storage in Android

Data Storage Options in Android

- **Shared Preferences**: Store small amounts of key-value pairs (e.g., user settings).
- **Internal Storage**: Store private data (files) on the device.
- **External Storage**: Store data that should be shared between apps or with the user (e.g., photos, downloads).
- **SQLite Database**: Store structured data in a relational database.

Data Storage in Android

Choosing the Right Storage

- Use **Shared Preferences** for settings and flags.
- Use **Internal Storage** for private, app-specific data.
- Use **External Storage** for media or data that needs to be accessed by other apps or users.
- Use **SQLite** for structured data that requires querying.

Android vs iOS

Android vs iOS:

- **Android**: Developed by Google, it is the most widely used mobile operating system in the world. Open-source and highly customisable.
- **iOS**: Developed by Apple, it is a closed-source operating system designed for Apple devices like iPhones and iPads.

Key Differences:

- **Android** is open-source, while **iOS** is proprietary.
- **Android** supports a wide variety of devices, while **iOS** is limited to Apple devices.

Android vs iOS

Android Development:

- **Languages**: Java, Kotlin (official language)
- **IDE**: Android Studio (official Integrated Development Environment)
- **Tooling**: Android SDK (Software Development Kit), Android APIs, Jetpack libraries.

iOS Development:

- **Languages**: Swift (preferred), Objective-C (older apps)
- **IDE**: Xcode (official IDE)
- **Tooling**: iOS SDK, Cocoa Touch libraries, SwiftUI for UI development.

See you in the next lecture…